Performance Testing

Performance testing evaluates howyoursystem behavesunder expectedand peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Date	22 October 2026	**Team ID**	Piyush-Balodiya- GenAI-2026
Project Name	CONSTRUCTAI (Multimodal Construction Intelligence)	**Maximum Marks**	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman v11 (API Contract Test Suite) & Locust v2.24 (Load & Stress Simulator)
Type of Testing	Load Testing, Stress Concurrency Testing, and API Contract Verification
Target Module / API	/api/v1/auth/login, /api/v1/planner/estimate, and /api/v1/qc/crack-detect
Test Environment	Cloud Production Staging (FastAPI on Render Web Service + Vercel Edge CDN Frontend)
Test Date	22 October 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	Virtual Users	Duration	Expected Outcome
1	Baseline Single User Session: Overview Hub telemetry & session check	1 User	60 sec	Avg Latency < 200ms, 100% HTTP 200 OK.
2	Automated Postman API Contract Suite: 6 Core endpoints & 24 assertions	1 User	1.39 sec	100% Passed (24/24), status code 200/201.
3	Standard Concurrency Load: Simultaneous staff authentication & JWT signing	25 Users	120 sec	Stable throughput > 25 req/sec, 0 error rate.
4	Peak Operational Stress: Concurrent BOQ slab calculations & crack analysis	50 Users	180 sec	Peak throughput > 50 RPS, zero 5xx server drops.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	**242 ms**	PASS	Fast async FastAPI ASGI loop
2	Response Time (Max / p95)	< 5 seconds	**368 ms**	PASS	Far below SLA threshold
3	Throughput (Req / sec)	> 20 req/sec	**58.4 req/sec**	PASS	Peak load handled via Uvicorn
4	Error Rate	< 1%	**0.00%**	PASS	0 failures out of 8,450 requests
5	CPU Utilization	< 80%	**34.2%**	PASS	Peak reached 46.8% under 50 VUs
6	Memory Utilization	< 80%	**29.5%**	PASS	Max RAM reached 35.1% (No leak)

Step 4: Observations & Analysis

Key Findings:

The performance benchmarks demonstrate that ConstructAI provides enterprise-level operational stability under sustained concurrent site workloads. FastAPI's native asynchronous request handling (ASGI via Uvicorn) along with decoupled compute workers ensured that 95% of incoming requests were resolved in under 368 milliseconds. The system achieved a peak throughput of 58.4 requests per second with an absolute 0.00% error rate across all load cycles.

Bottlenecks Identified:

During preliminary stress ramp-up beyond 60 concurrent requests, synchronous image decoding inside the crack diagnostics pipeline produced minor event-loop delays (~550ms latency spikes). Furthermore, SQLite write locks on the staff provisioning registry created a transient wait state during bulk account generation.

Optimization Steps Taken:

1. **Asynchronous Worker Offloading:** Converted heavy image processing and Groq LPU multimodal requests to background async coroutines (asyncio.to_thread), preventing event-loop blocking.
2. **Connection Pooling & WAL Mode:** Enabled SQLite Write-Ahead Logging (WAL mode) to permit concurrent reads while transactions write, dropping lock contention to zero.
3. **Frontend Decoupling:** Decoupled heavy React state trees between the Project Planner and Staff Provisioning modules, reducing frontend hydration and client-side memory footprint.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (Postman API Collection Runner, Locust load test concurrency dashboard):



